

Open in app ↗

Sign up

Sign in

Medium

 Search

Android

+

Java

+

Nfc

+

Ntag 424

+

Sdm

+

Demystify the Secure Dynamic Message with NTAG 424 DNA NFC tags (Android/Java) Part 1

12 min read · Jun 19, 2024



AndroidCrypto

Follow



Listen



Share

This article will help you in understanding the “mysterious” **Secure Dynamic Message** (“SDM”) or **Secure Unique Number** (“SUN”) feature, available in NXP’s NTAG 424 DNA NFC tags.

This is **part 1** of the article series about the **Secure Dynamic Message** (“SDM”) and gives a lot of information about the complete data management, **part 2** is the practical implementation in an Android app.

The NTAG 424 DNA NFC tag family is — when just reading the technical facts — unspectacular. It comes with 3 predefined files with a total of 416 bytes storage capacity. *It is compliant to NFC Forum Type 4 Tag and ISO/IEC 7816–4 with a Capability Container (CC) file to specify the NFC Forum tag operation, an NDEF file as well as an extra data file to protect sensitive content.*

The tags got an EAL4 certification for both Hardware and Software, have five customer defined AES 128-bit keys including key versions, an optional Random ID for enhanced privacy and the well known 3-pass mutual authentication. Besides the standard AES-128 implementation, NTAG 424 DNA also offers an alternative AES-based protocol for authentication and secure messaging using a Leakage Resilient Primitive, or LRP, a wrapper around the AES cryptography to enhance side-channel attack resistance. (see the [data sheet](#) for more details)

But the most fascinating feature — in my eyes — is the SUN or SDM feature: when enabled you tap the tag against a smartphone that recognizes the provided **NDEF message**. This NDEF message should be of NDEF type “**URL-Link**” and the smartphone tries to contact the server. The server is then able to read the data and verify that the data was not tampered or changed.

This feature was not introduced with this tag — the SDM/SUN feature was offered with the **Mifare DESFire EV3** tags the first time, but they are much more expensive than the **NTAG 424 DNA** tags (of course, they have a lot of more functionality and storage capacity). The reason why there is nothing to read about the SDM feature with DESFire tags is easy: most of the technical details regarding this feature is hidden under a “Non Disclosure Agreement” documentation, meaning that only companies that sign this agreement are allowed to read the papers.

There is just one hiccup when trying to work with NTAG 424 DNA tags — to run the most important commands you need to implement the “**AuthenticateEV2**” or “**AuthenticateLRP**” function. The good news is: there is a Java library available that does most of the work for us; it is the “**ntag424-java**” library, written by **Jonathan Bartlett**.

Update October 12th 2025

As I received a lot of requests, I added an article that gives in detail “**A comprehensive overview of all keys for the NTAG424 NFC chip**”.

In my GitHub repository I’m providing a “ready to use” (pre-build, debug) app that runs the examples I’m describing here (see the link at the end of this article).

Before I’m going into the details I’d like to give you an example of the feature. As the SDM feature is based on an URL-type NDEF message I’m starting with this **URL template**:

```
https://sdm.nfcdeveloper.com/tagpt?uid=*****&ctr=*****&cmac=*****
```

If you click on the link https://sdm.nfcdeveloper.com/tagpt?uid=*****&ctr=*****&cmac=***** you are directed to a “Secure Dynamic Messaging Backend Server Demo” that states: **400 Bad Request: Failed to decode parameters.**

The reason for the failure is easy: you see a lot of “*” characters in the link that are working as placeholders for the real NDEF message (data). After enabling the feature the link would look like <https://sdm.nfcdeveloper.com/tagpt?uid=049F50824F1390&ctr=000001&cmac=2446E527C37E073A> — now we receive a positive “**Cryptographic signature validation**” result from the backend server:

Secure Dynamic Messaging Backend Server Demo

Cryptographic signature validated.

Encryption mode: AES

NFC TAG UID: **049f50824f1390**

Read counter: **1**

Let's have a closer look to the data send to the server:

- “https://sdm.nfcdeveloper.com” is “our” server that should validate our tag data
- “tagpt” is the header for **plaintext tag data** (for other headers see <https://sdm.nfcdeveloper.com/>)
- “uid” is the **unique tag UID** (“049f50824f1390”)
- “ctr” is the **read counter** (“1”)
- “cmac” is a **digital signature on the data** of “UID” and “read counter” (“2446E527C37E073A”)

The tag is “inserting” the data dynamically in the placeholders without any additional programming, and you do not need any custom app to run the validation, just a server like the sdm.nfcdeveloper.com one and a modern Android or iOS smartphone with internet access.

A note regarding the backend server I'm using in this tutorial:

sdm.nfcdeveloper.com is provided by "NFC Developer" (Michał Leszczyński) in the GitHub repository <https://github.com/nfc-developer/sdm-backend>. The service can validate the tag data of NTAG 424 DNA tags that operate with **default AES keys only** (meaning AES-128 keys with 16 '00h' bytes. In the progress of the tutorial we are going to change ("personalize") the keys and the server validation will fail. The server is also not been able to process plaintext messages with UID or Counter only (but it works with UID and Counter in a message as show above).

Activate the SDM feature

To activate the SDM feature you need to follow a 2 step workflow:

Step 1: upload ("write") a "Capability Container" dataset to file 1 and upload ("write") the NDEF URL template to file 2 of the NTAG 424 DNA tag

Step 2: change the file settings of file 2 to enable the SDM feature depending on the matching fields. The position of the fields are set by using "offset" parameters, together with some additional informations about "Keys".

Step 1: Prepare the tag by uploading a Capability Container

The NTAG 424 DNA tag is sold with a default Capability Container (CC) that defines e.g. the maximum amount of data available for a NDEF message and the reading and writing access conditions (for more details see the PDF [specification](#)). Unfortunately the write access is free available, so we need to use a CC that sets the access condition to "Read Only".

The CC I'm using is this 32 bytes long sequence:

```
"001720010000FF0406E104010000FF0506E1050080828300000000000000000000"
```

Step 2: Write a NDEF message and change the file settings

As the tag should operate without a specific app the NDEF message stored in file has to be of "URL"-type. The library takes an URL-string and writes the data as structured NDEF message. At this point you need to define which data should be part of the NDEF message — it is grouped into 3 different types.

Group 1: Plaintext data

The plaintext data is one or both of these data fields:

- The 7 bytes long tag UID (“Unique Identifier”)
- The 6 bytes long “Read Counter”

Mandatory is a third field called “CMAC” that is an eight bytes long cryptographic signature based on a specified application key (key numbers are from 0 to 4). The CMAC printed in the message is the shortened version of the “full” 16 bytes long CMAC.

A short note on the “UID”: for security reasons the tag can get setup for a **Random UID**. This is a one-time setting and cannot get reversed, so my app isn’t setting this feature (but the library has a function for that). When the feature is active the tag is responding just a 4 bytes long random UID (starting with 08h) after the first communication. The real (7 bytes long) UID is returned only after a successful authentication, but when the UID is setup in the NDEF message it is presented there as well. For security reasons I strongly recommend to use the “Random UID” feature with Encrypted data only (see next part).

Above I have analyzed an example Link, that I’m publishing here again for completeness:

<https://sdm.nfcdeveloper.com/tagpt?uid=049F50824F1390&ctr=000001&cmac=2446E527C37E073A>

Group 2: Encrypted data

The encrypted data is named “PICC” data in the documentations and has these 2 field options (again: one or both of them):

- The 7 bytes long tag UID (“Unique Identifier”)
- The 6 bytes long “Read Counter”

As the “plaintext” data is encrypted we need to provide the key number of one application key (see group 1). Mandatory again is a third field called “CMAC” (see group 1, the digital signature).

This is the more secret version (compared to group 1) as anyone with a smartphone is being able to see the encrypted data only, especially when the “Random UID” feature is active.

Here is an example NDEF link with encrypted PICC data, containing the tag UID and Read Counter:

[https://sdm.nfcdeveloper.com/tag?
picc_data=EF963FF7828658A599F3041510671E88&cmac=94EED9EE65337086](https://sdm.nfcdeveloper.com/tag?picc_data=EF963FF7828658A599F3041510671E88&cmac=94EED9EE65337086)

Get AndroidCrypto's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

☐ Remember me for faster sign in

Here I'm structuring the data that are transmitted:

The backend server URL:

`https://sdm.nfcdeveloper.com/`

The endpoint API for encrypted data ("tag"):
`tag?`

The field tag for the encrypted PICC data:
`picc_data=`

The encrypted PICC data:
`EF963FF7828658A599F3041510671E88`

The field tag for the CMAC:
`&cmac=`

The CMAC data:
`94EED9EE65337086`

After decryption on the backend server side these data are visible (just click on the above link:

```
Encryption mode: AES
PICC Data Tag: c7
NFC TAG UID: 04de5f1eacc040
Read counter: 61
```

The tag is adding some random padding data to the tag UID and Read Counter data, so the next transmitted link will look like total different.

Group 3: Encrypted data with encrypted file data

The data fields in Group 2 (encrypted UID and/or Counter) are dynamically given out, but sometimes you need to add some static data for this specific tag. Think of data like an “Issuing timestamp” or a “Personal Identification Number” that need to be transferred along within the NDEF message. This is available as well, additionally to the data in group 2. The encrypted file data is stored in the NDEF file between the (encrypted) PICC data and the CMAC.

This feature is amazing, but you should be aware that the NDEF message gets much longer, and some NFC card readers have a limited buffer. A “too long” message may produce a buffer overflow error !

Here is the example NDEF URL-link for an encrypted PICC data, combined with encrypted file data:

[https://sdm.nfcdeveloper.com/tag?
picc_data=4E8D0223F8C17CDCCE5BC24076CFAA0D&enc=B56FED7FF7B23791C068
4F17E117C97450723BB5C104E809C8929F0264CB99F9969D07FC32BB2D11995AEF8
26E355097&cmac=5FD76DE4BD942DFC](https://sdm.nfcdeveloper.com/tag?picc_data=4E8D0223F8C17CDCCE5BC24076CFAA0D&enc=B56FED7FF7B23791C0684F17E117C97450723BB5C104E809C8929F0264CB99F9969D07FC32BB2D11995AEF826E355097&cmac=5FD76DE4BD942DFC)

Here I’m structuring the data that are transmitted:

```
Backend Server:
https://sdm.nfcdeveloper.com/

The endpoint API for encrypted data ("tag"):
tag?

The field tag for the encrypted PICC data:
picc_data=

The encrypted PICC data:
```


the tag and its security features. The NTAG 424 DNA is coming with **5 application keys** that are **AES-128** (16 bytes) long and filled with 16 '00h' bytes ("default keys").

Even with the usage of default keys every (encrypted) NDEF message will look like different.

Keys used within this tutorial

A NTAG 424 DNA tag with fabric settings has these access conditions for the files 01 and 02 (as file 03 is not used within this tutorial I leave the settings out here). The most used is key number 0 as it is the "master application key" and is responsible for the write access to file 01 and change the access conditions on all files (in default settings).

Access Settings for file 01

As per data sheet page 12 the file 01 can get **read** without any authentication (access condition is 'key' Eh = 14d, meaning free access). For **writing** to the file we need to authenticate with key 0 (write or read&write access) before writing. If we want to change the file settings we need to authenticate with key 0 as well. These access rights apply when trying to read the content using the read commands as per data sheet. When you access the file by NDEF these settings are not active.

Access Settings for file 02

The settings for file 02 are easier as file 01: you can read from and write to the file without any authentication (again the 'key' Eh = 14d, meaning free access, is used). For changing the file settings we need an authentication with key 0.

Personalization of the tag

An important part within the workflow is to **personalize** the tag, meaning to change the file settings above (change the write access key of file 02 to e.g. key 0) and — more important — **change the default keys itself to individual keys**. This includes all application keys, even if they are not used in your particular project. you should consider to change the tag settings to a random tag UID (see below for an explanation) as this prevents any tracking of tags in usage. The last personalization step would be the change of authentication scheme from AES to LRP. The change to **random UID** and LRP are supported by our library, but are not part of this tutorial.

Random Tag Unique Identifier (UID)

A NFC tag reader is polling for any card ("is here any card?") by transmitting some commands. When a card is in the electrical field it gets powered up and answers like ("I'm here"). In the next step the reader and tag are talking about the protocol the

card likes to get the commands. In this phase the reader asks the card for his “number” or “tag UID”. A NTAG 424 DNA has a 7 bytes long UID that was given during factory setup of the tag. On default settings this UID is transmitted to the reader. The problem with this is that anyone can track this specific UID (“privacy concerns”). To solve this problem the tag can get setup to generate a random UID (usually 4 bytes long, starting with “08h”). Each time the tag is tapped to the reader it presents a new UID so an easy tracking of the card holder is not possible.

When the tag is in Random UID mode the tag will present its “real” UID (not the random one) in the NDEF message. If privacy is important to your business case you should consider to use the encrypted PICC data together with random UID mode.

Please note: the setting of random UID mode is irreversible, for that reason I’m not setting this mode in this tutorial.

Key Diversification

The key diversification is important to **increase the security** of your complete system. The diversification changes the key subject to other parameter(s). The NTAG 424 DNA tags were created by NXP and they defined a key diversification depending on three parameters (Link to the application note [Symmetric Key Diversification AN10922](#)):

- a “system identifier” — this is an identifier for your complete infrastructure
- an “application identifier” naming your specific application
- a tag specific parameter, the “real tag UID”

Now we are putting all parameters together, we need a “master key” that is the base for the diversification. The NTAG 424 DNA library is supporting this diversification and in the end we have a key that is valid for the specific tag only, so an attacker is not been able to have access to other tag data.

The following keys are setup during step 2 of the tag preparation (write NDEF template and change file settings):

Key used for CMAC calculation

One of the application keys has to be defined as the key that the tag uses for the calculation of the CMAC. In the documentations this key is named as “**SDM File Read Permission**”. In my examples I’m using the (default) Key number 2 for this

task, except in “EncryptedFileSunCustomKeys” where the “Custom Key Number 4” is used:

```
byte[] APPLICATION_KEY_4 = Utils.hexStringToByteArray("A400000000000000000000000000000000");
```

The above key could get diversified (see description above).

Key used for Encrypted PICC data

If we are using encrypted PICC data (tag UID and/or Read Counter) the data is encrypted with a key named as “**SDM Meta Read Permission**”; I’m using the (default) Key number 2, except in “EncryptedFileSunCustomKeys” where the “Custom Key Number 3” is used:

```
byte[] APPLICATION_KEY_3 = Utils.hexStringToByteArray("A300000000000000000000000000000000");
```

This key cannot get diversified as you would need the tag UID before you can decrypt the data that includes the tag UID.

Key used for Encrypted File data

The tag will encrypt the File data with the same key as used for calculating the CMAC (that is the SDM File Read Permission key).

At this point I explained all the stuff to understand an implementation in a “real” Android app, please step over to **part 2** of my tutorial:

Demystify the Secure Dynamic Message with NTAG 424 DNA NFC tags (Android/Java) Part 2

+

+

+

+

+